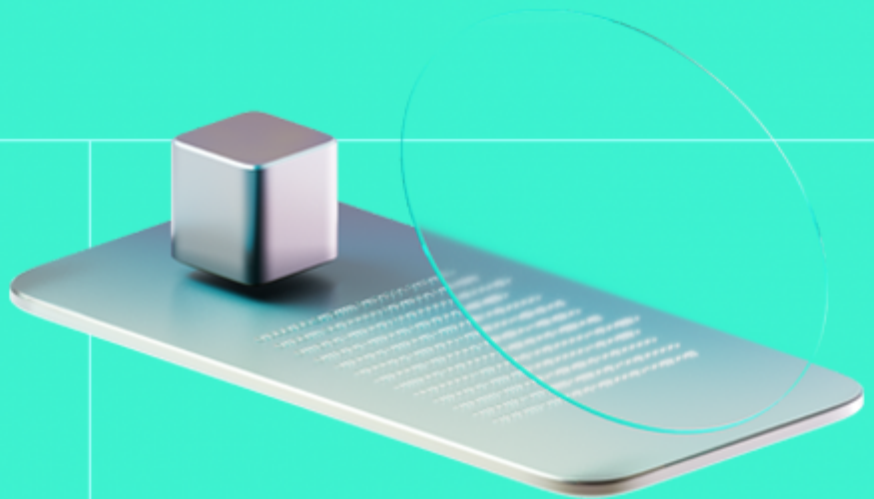




Smart Contract Code Review And Security Analysis Report

Customer: Echo Protocol

Date: 28/08/2025



We express our gratitude to the Echo Protocol team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

Echo Bridge protocol is an IOTA Ethereum bridge solution. The protocol enables cross-chain bridging and communication between the IOTA blockchain and EVM compatible chains. Its primary goal is to support bidirectional token bridging, allowing users to move assets seamlessly between chains using a validator committee for message verification. The system ensures trust-minimized operations through threshold-based signature approval, nonce-based replay protection, and committee governance. ERC20 tokens can be locked on the EVM side and minted as wrapped assets on IOTA, and vice versa.

Document

Name	Smart Contract Code Review and Security Analysis Report for Echo Protocol
Audited By	Turgay Arda Usman, Khrystyna Tkachuk
Approved By	Ataberk Yavuzer
Website	https://www.echo-protocol.xyz/
Changelog	04/08/2025 - Preliminary Report 28/08/2025 - Final Report
Platform	IOTA
Language	Move
Tags	Bridge, Signatures
Methodology	https://hackenio.cc/sc_methodology

Review Scope

Repository	https://github.com/echo-proto/bridge-iota
Initial Commit	686e118
Final Commit	89ffe353

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

11	6	5	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	2
Medium	4
Low	3

Vulnerability	Severity
F-2025-11913 - Token Supply Inflation via Improper Refund Logic in approve_token_transfer and claim_token	High
F-2025-11959 - The Committee is Static and Cannot be Rotated	High
F-2025-11856 - Mismatched Array Lengths in Admin Messages Can Lead to State Corruption or Denial of Service	Medium
F-2025-11960 - Blocklisted Status is Not Persistent Across Committee Rotations	Medium
F-2025-11997 - Ineffective Revocation of Submitter Privileges	Medium
F-2025-12021 - Artificially Low Asset Price Leads to Rate-Limiter Bypass	Medium
F-2025-11854 - Token ID Collision Allows State Corruption	Low
F-2025-11904 - Unchecked Read in peel_u64_be Leads to Transaction Abort and Denial of Service	Low
F-2025-11995 - Lack of Zero-Value Validation in update_route_limit() Allows Setting Invalid Rate Limits	Low
F-2025-11855 - Inconsistent Logic Due to Partial Redundancy Check	Info
F-2025-11994 - Commented Code Parts	Info

Documentation quality

- Functional requirements are partially provided.
 - Use cases and project purpose are not provided.
 - Features are partially provided.
- Technical description is not provided.
 - Key function descriptions are partially provided.
 - Architecture is not described.

Code quality

- The code mostly follows best practices and style guides.
 - See informational findings for more details.

Test coverage

- Deployment and basic user interactions are covered with tests.
- Negative cases coverage is missed.
- Interactions by several users are not tested thoroughly.

Table of Contents

System Overview	6
Privileged Roles	6
Potential Risks	7
Findings	8
Vulnerability Details	8
Disclaimers	41
Appendix 1. Definitions	42
Severities	42
Potential Risks	42
Appendix 2. Scope	43

System Overview

Echo Bridge is a bridge protocol that supports asset bridging from EVM chains to IOTA and vice versa. It includes the following modules:

- `bridge` - the main contract serves as the central entry point for all bridge functionality. It handles the configuration of critical data and provides governance mechanisms for executing system-level configuration transactions;
- `committee` - responsible for managing the committee, this component tracks active members and maintains a blacklist for excluded participants;
- `chain_ids` - responsible for storing hardcoded chain IDs and managing the addition of supported bridge routes;
- `crypto` - a utility module providing a single function to convert an ECDSA public key into an Ethereum address;
- `ibtc` - defines and initializes the `iBTC` token as a bridged Bitcoin asset on IOTA. Sets up treasury, metadata, and finalizes bridge initialization;
- `limiter` - this module enforces rate limits on value transferred per route using a 24-hour ring buffer. It tracks hourly outflows, maintains a rolling sum, and checks if new transactions exceed the configured limit. Limits and asset prices are adjustable via a dedicated function;
- `message_types` - contains hardcoded message type codes, and getters for them;
- `message` - this module handles encoding and decoding of bridge messages. It also defines constants for pause and unpause message types and includes logic for determining the voting power required to execute each message type;
- `treasury` - manages separate treasuries for each supported token and tracks their approximate prices for rate-limiting purposes. It supports minting, burning, price updates, and adding new tokens. Token support, once added, cannot be removed.

Privileged roles

- The bridge `AdminCap` can update the `fee_recipient` and `submitters`, and is responsible for registering and initializing the committee. Additionally, the admin can migrate the bridge and admin capability versions.
- The `TreasuryCap` is responsible for adding new tokens to the bridge. Tokens are first placed in a waiting room and require approval through committees.
- The submitter is responsible for triggering the `approve_token_transfer` function to approve token bridging.
- The committees is responsible for executing system messages and signing bridge transaction approval requests.

Potential Risks

- The audit does not cover all code in the repository. Contracts outside the audit scope may introduce vulnerabilities, potentially impacting the overall security due to the interconnected nature of smart contracts. The bridge protocol is designed to transfer assets from EVM-compatible chains to the IOTA chain. However, the audited scope included only the Move-based implementation on the IOTA side. The EVM-side bridge logic was not part of the review and may introduce security concerns that could affect the overall integrity of the bridging process.
- Granting significant control to whitelist users without adequate checks leads to unpredictable operational disruptions, impacting user confidence and system stability.
- The current version of the bridge does not support bridging the IOTA native asset. This limitation may restrict functionality or interoperability expectations for users intending to transfer IOTA tokens across chains.
- The `notional_amount`, which represents the USD value of assets, is set and updated manually by committee governance. This value is not backed by an external price oracle and lacks any time-based guarantees for regular or accurate updates, potentially leading to stale or incorrect pricing.

Findings

Vulnerability Details

F-2025-11913 - Token Supply Inflation via Improper Refund Logic in approve_token_transfer and claim_token - High

Description:

The bridge's refund mechanism, designed to return funds to a user after a failed cross-chain transfer, contains a critical flaw that leads to permanent token supply inflation. When a refund is processed on the IOTA chain, the system mints the user's original gross token amount, failing to account for the non-refundable fee that was deducted and paid out during the initial transfer request. Because more tokens are minted in the refund than were originally burned, each failed transaction inflates the circulating supply of the bridged asset, violating the fundamental 1-to-1 asset backing of the bridge.

The issue originates from a discrepancy between the amount of tokens burned and the amount recorded for a potential refund. The issue unfolds across three stages of the transaction lifecycle: initiation, approval for refund, and claiming the refund.

- Transaction Initiation (`send_token`): When a user initiates a transfer, the `send_token` function correctly deducts a fee. It transfers this fee to a recipient and then burns the net amount (`original_amount - fees`). However, it simultaneously creates and stores a `BridgeMessage` in `token_transfer_records` that contains the gross `original_amount`.

```
public fun send_token<T>(...) {  
    ...  
    if (fees > 0) {  
        let fees_balance = balance::split(&mut token_balance, fees);  
        let fee_coin = coin::from_balance(fees_balance, ctx);  
        transfer::public_transfer(fee_coin, inner.fee_recipient);  
    };  
    let amount = token_balance.value();  
    let message = message::create_token_bridge_message(  
        ...  
        original_amount  
    );  
    let burn_coin = coin::from_balance(token_balance, ctx);  
    inner.treasury.burn(burn_coin);  
    inner.token_transfer_records.push_back(  
        ...  
    );  
}
```



```

        message.key(),
        BridgeRecord { message, ... },
    );
    ...
}

```

- Refund Approval (`approve_token_transfer`): If the cross-chain transfer fails, the documented refund process begins. The committee provides signatures for the exact BridgeMessage stored in `token_transfer_records`. The `approve_token_transfer` function marks this record as verified, greenlighting it for a refund claim.
- Claiming the Refund (`claim_token` / `claim_and_transfer_token`): The user (or an admin) then calls one of the claim functions, which executes the `claim_token_internal` logic. This function reads the amount from the approved message payload and mints that quantity for the user.

```

fun claim_token_internal<T>(...) {
    ...
    let record = &mut inner.token_transfer_records[key];
    let token_payload = record.message.extract_token_bridge_payload();

    let amount = token_payload.token_amount();

    let token = inner.treasury.mint<T>(amount, ctx);
    ...
    record.claimed = true;
    ...
}

```

The fee, having already been paid out in first bullet point, is never clawed back. The result is a net increase in the total token supply equal to the fees for every refunded transaction.

This issue creates a slow but steady drain on the bridge's solvency. Each refunded transaction introduces unbacked tokens into the IOTA ecosystem, breaking the essential 1-to-1 asset peg.

Assets:

- `./bridge.move` [<https://github.com/echo-proto/bridge-iota>]

Status:

Fixed

Classification

Impact Rate:

4/5

Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	High

Recommendations

Remediation: Consider updating the message crafting logic in the `send_token()` function to use the net token amount after fee deduction. This ensures consistency between the actual transferred value and the metadata used in the bridging process, preventing over-release and preserving the integrity of the destination bridge's token supply.

```
// create bridge message
let message = message::create_token_bridge_message(
    inner.chain_id,
    bridge_seq_num,
    address::to_bytes(ctx.sender()),
    target_chain,
    target_address,
    target_token_id,
    amount // <--
);
```

Resolution: In commit `d56f48e1`, the message now holding the net amount for ordinary transfers and the refund flow burning the fee before restoring the gross amount, the one-to-one backing between chains is preserved in every path.

Evidences

PoC

Reproduce:

1. The `send_token()` function is used to initiate a cross-chain token transfer bridging tokens from the IOTA chain to the EVM chain.
2. The user selects a `target_chain` route that applies a non-zero fee, specifically a 10% fee rate.
3. The user requests to bridge `1000` units of the token.
4. Based on the configured fee, `100` units are deducted and transferred to the `fee_recipient` on the source chain, while `900` units are effectively burned as the net bridged amount.

5. A bridging message is constructed to be signed by the bridge committee and written to the `token_transfer_records` mapping. However, it incorrectly includes the original amount of `1000` units instead of the actual burned amount of `900` units on the source chain.
6. The bridge transaction is failed to execute on the destination chain. User try to do refund via `claim_token` function on the source chain.
7. On the source chain, the bridge contract processes the `token_transfer_records` for the bridge message and unlocks `1000` tokens to the user despite only `900` should be minted since the `100` was transfer to the fee recipient in the origin transaction once the `send_token()` was called.

Message that is crafted on the source chain and passed to the `token_transfer_records` mapping:

```
let message = message::create_token_bridge_message(  
    17, // Source chain ID: IOTA  
    1, // Sequence number  
    0x0userAddress, // Sender address on IOTA  
    1, // Target chain ID: Ethereum  
    0xuserAddressOnEthereum, // Recipient address on EVM  
    1, // Target token ID  
    1000 // Incorrect: pre-fee amount used instead of 900  
);
```

[F-2025-11959](#) - The Committee is Static and Cannot be Rotated - High

Description:

The bridge's security relies on a committee of validators who approve and sign messages. The `create_bridge_committee` function is intended to initialize this committee and, implicitly, to refresh or rotate it over time by calling the internal `try_create_next_committee` function. This rotation is essential to ensure the committee's voting power accurately reflects the mainnet's validator stake distribution in any given epoch.

Due to a combination of checks in `bridge.move` and state-clearing logic in **`committee.move`**, the bridge committee can only be created once and can never be updated or rotated thereafter. This flaw permanently locks control of the bridge to the founding members, completely decoupling its security from the underlying economic stake of the mainnet. An old, low-stake committee could retain indefinite control over all bridged assets.

The operational lifecycle of the committee is irreparably broken by logic in two separate modules. First, the `bridge::init_bridge_committee` function, which is the entry point for committee creation, is guarded by a check that only permits execution if the active committee is empty:

```
fun init_bridge_committee(...) {  
    let inner = load_inner_mut(bridge);  
    if (inner.committee.committee_members().is_empty()) {  
        inner.committee.try_create_next_committee(...)  
    }  
}
```

Second, upon a successful committee creation, the `committee::try_create_next_committee` function erases the list of all registered candidates:

```
if (stake_participation_percentage >= min_stake_participation_percentage) {  
    self.member_registrations = vector::empty();  
    self.members = new_members;  
    ...  
}
```

This creates a one-way trip:

1. Registration is only possible when the committee is empty.

2. Committee creation is only possible when the committee is empty.
3. The first successful creation populates the committee, making it non-empty.
4. As a result, both the registration and creation functions are permanently disabled.

This can lead to a situation where the founding committee members have indefinite control, regardless of their future stake in the Iota network. They cannot be voted out or replaced.

Assets:

- `./committee.move` [<https://github.com/echo-proto/bridge-iota>]

Status:**Fixed**

Classification

Impact Rate:	4/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	High

Recommendations**Remediation:**

The logic should be refactored to support a lifecycle of rotation as follows:

1. Remove the `if (inner.committee.committee_members().is_empty())` check from `bridge::init_bridge_committee`. The function should be able to execute on an existing committee to initiate a rotation.
2. Remove the line `self.member_registrations = vector::empty()` from `committee::try_create_next_committee`. The list of registered candidates should persist across epochs, allowing for a continuous pool of potential members.
3. The `committee::register` function should be modified to allow new candidates to be added even after a committee is active, enabling the validator pool to grow over time.

Resolution:

Fixed in `89ffe35`, the initialization check in `bridge::init_bridge_committee` was removed, allowing committee rotation to be triggered on an existing committee. The `committee::register` function was also updated so new candidates can join while a committee is active. However,

candidate registrations are still cleared each epoch in `committee::try_create_next_committee`, the design now supports reinitializing the registration list each epoch with the ability to re-add addresses already present in the members list.

Evidences

PoC

Reproduce:

The test begins by running `create_bridge_default()`, which sets up the bridge and, most importantly, **forms the very first active committee**. This puts the contract in a state where a committee rotation would be the next logical administrative action.

It then simulates an administrator initiating a new transaction with the specific goal of rotating the committee. It takes control of the Bridge object and the AdminCap needed to perform this privileged operation.

The test calls the public function `bridge.create_bridge_committee(...)`. This is the one and only function available to an administrator that is intended to update or rotate the committee members and their voting power.

Immediately after calling the function, the test queries for a `CommitteeUpdateEvent` within the same transaction. This event is the definitive on-chain proof that a new committee was successfully formed. If the rotation had worked, this event would have been emitted.

PoC Code:

```
#[test_only]
module bridge::committee_rotation_tests;

use bridge::bridge::{Bridge, AdminCap};
use bridge::bridge_env::{create_env, BridgeEnv};
use bridge::committee::{Self as committee, CommitteeUpdateEvent};
use iota::vec_map;
use iota::event;
use iota::test_scenario;

#[test]
fun test_committee_rotation_impossible() {
  let mut env = create_env();
  env.create_bridge_default();
```

```

let sender = env.submitter();
let scenario = env.scenario();
scenario.next_tx(sender);

// Take the shared Bridge object and the admin capability.
let mut bridge = scenario.take_shared<Bridge>();
let admin_cap = scenario.take_from_address<AdminCap>(sender);

let active_committee_powers = vec_map::empty<vector<u8>, u64>();

bridge.create_bridge_committee(
    &admin_cap,
    active_committee_powers,
    50, // min_stake_participation_percentage (arbitrary)
    scenario.ctx(),
);

iota::test_scenario::return_shared(bridge);
iota::test_scenario::return_to_sender(scenario, admin_cap);

let rotation_events = event::events_by_type<CommitteeUpdateEvent>();
assert!(rotation_events.length() == 0, 99999);

// Clean up.
env.destroy_env();
}

```

Results:

```

[ PASS    ] bridge::committee_rotation_tests::test_committee_rotation_impossible
Test result: OK. Total tests: 1; passed: 1; failed: 0
Total number of linter warnings suppressed: 1 (unique lints: 1)

```

[F-2025-11856](#) - Mismatched Array Lengths in Admin Messages Can Lead to State Corruption or Denial of Service - Medium

Description:

The system relies on privileged, off-chain administrators to configure its core parameters, such as which tokens are supported and what routes are active. This is accomplished via signed messages created by the `create_add_tokens_on_iota_message` and `create_add_routes_on_iota_message` functions in the `bridge::message` module. These functions accept several parallel vectors of data (e.g., `token_ids`, `token_type_names`, `token_prices`) which are then encoded into a message payload. This message is subsequently processed on-chain by the `execute_add_tokens_on_iota` and `execute_add_routes_on_iota` functions, which decode the payload and update the bridge's state.

The message creation functions (`create_add ...`) fail to validate that the parallel vectors provided as input have identical lengths. A malicious or mistaken administrator can therefore craft a syntactically valid message with mismatched array lengths (e.g., providing 3 token IDs but only 2 prices). When the bridge attempts to process this malformed message, the transaction will abort due to an out-of-bounds error, leading to a Denial-of-Service (DoS) that prevents any administrative updates. This could trap the bridge in a misconfigured or paused state indefinitely.

The issue exists within the `bridge::message` module. The two primary functions for creating administrative messages do not enforce critical data invariants.

- **`**create_add_tokens_on_iota_message**`**: This function takes three vectors that should correspond to each other index-wise, but it never checks if they are the same size.

```
public fun create_add_tokens_on_iota_message(  
    source_chain: u8,  
    seq_num: u64,  
    native_token: bool,  
    token_ids: vector<u8>,  
    type_names: vector<String>,  
    token_prices: vector<u64>,  
) : BridgeMessage {  
    chain_ids::assert_valid_chain_id(source_chain);  
    let mut payload = bcs::to_bytes(&native_token);  
    payload.append(bcs::to_bytes(&token_ids));  
    payload.append(bcs::to_bytes(&type_names));  
    payload.append(bcs::to_bytes(&token_prices));  
}
```



```
}
```

- `create_add_routes_on_iota_message`: This function has the same flaw across five parallel vectors.

An attacker or compromised admin could call these functions with inconsistent arguments. For example:

- `token_ids`: [1, 2, 3] (length 3)
- `type_names`: ["BTC", "ETH", "USDC"] (length 3)
- `token_prices`: [50000, 3000] (length 2)

A `BridgeMessage` would be successfully created. However, when this message is processed by `execute_add_tokens_on_iota` in `bridge` logic will fail. The processing function does contain assertions, but it also consumes the vectors by popping elements from the end in a loop.

```
fun execute_add_tokens_on_iota(inner: &mut BridgeInner, payload: AddTokenOnIo
ta) {
    ...
    // Make sure token data is consistent
    assert!(token_ids.length() == token_type_names.length(), EMalformedMessage
Error);
    assert!(token_ids.length() == token_prices.length(), EMalformedMessageErro
r);
    while (token_ids.length() > 0) {
        let token_id = token_ids.pop_back();
        let token_type_name = token_type_names.pop_back();
        let token_price = token_prices.pop_back(); // Fails here on 3rd iterat
ion if asserts were removed
        inner.treasury.add_new_token(token_type_name, token_id, native_token,
token_price)
    }
}
```

Because the check exists only on the receiving end, the malformed message can still be broadcast and submitted, causing any attempt to apply it to fail.

This can lead to a **Denial of Service** against the bridge's administrative functions.

Assets:

- `./message.move` [<https://github.com/echo-proto/bridge-iota>]

Status:

Accepted

Classification

Impact Rate:	3/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: Add array length validations to `create_add_tokens_on_iota_message` and `create_add_routes_on_iota_message` functions

Resolution: The client has acknowledged the finding with the following statement:

We have decided not to modify this part of the code. Our implementation is based on the Sui bridge code, which also does not include such validation. Reference link: <https://github.com/MystenLabs/sui/blob/main/crates/sui-framework/packages/bridge/sources/message.move#L234>

F-2025-11960 - Blocklisted Status is Not Persistent Across Committee Rotations - Medium

Description:

The `execute_blocklist` function is a critical security tool that allows governance to revoke a committee member's signing rights by setting their blocklisted flag to true. This is used to respond to malicious activity or compromised keys.

A committee member's blocklisted status is not persistent. If a committee rotation were possible, any previously blocklisted member would be automatically reinstated as a fully trusted member in the new committee. This flaw would force governance to re-apply blocklists for known bad actors after every single epoch, an operationally fragile and unrealistic security practice.

When `try_create_next_committee` constructs the new set of committee members, it hardcodes the blocklisted status of every new member to false, without checking their prior status.

```
let member = CommitteeMember {  
    bridge_pubkey_bytes: *bridge_pubkey_bytes,  
    voting_power: (voting_power as u64),  
    // ...  
    blocklisted: false,  
};
```

This means that a validator who was blocklisted for malicious behavior in epoch N would, upon the next successful committee rotation in epoch N+1, have their blocklisted flag reset to false. They would rejoin the committee with full signing privileges.

This makes impossible to permanently ban a malicious actor. The ban only lasts for the duration of the current epoch.

Assets:

- `./committee.move` [<https://github.com/echo-proto/bridge-iota>]

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation:

1. Modify the `BridgeCommittee` struct to include a persistent `vec_set` or `vec_map` of blocklisted public keys (or addresses).
2. When creating a new committee member in `try_create_next_committee`, the logic should check if the candidate's key/address exists in this persistent blocklist set. If it does, the new member's blocklisted field should be initialized to true.

Resolution:

Fixed in `d60e18e`, added a check to verify if the member already exists, ensuring that the current `blocklisted` status is preserved when updating the record:

```
let mut blocklisted = false;
if (self.members.contains(bridge_pubkey_bytes)) {
    let member = &self.members[bridge_pubkey_bytes];
    blocklisted = member.blocklisted;
};

let member = CommitteeMember {
    bridge_pubkey_bytes: *bridge_pubkey_bytes,
    voting_power: (voting_power as u64),
    // http_rest_url: registration.http_rest_url,
    blocklisted: blocklisted,
};
```

[F-2025-11997](#) - Ineffective Revocation of Submitter Privileges - Medium

Description:

The `bridge.move` contract designates certain off-chain actors as "submitters." These entities are responsible for relaying messages, such as transfer approvals, to the bridge. The contract owner, via an `AdminCap`, can use the `update_submitter` function to grant or revoke these privileges. This function maintains a `VecMap<address, bool>` where the boolean is intended to represent the submitter's active status.

The access control mechanism for submitters is flawed. The check within `approve_token_transfer` only verifies that a transacting address exists as a key in the submitters map, but completely ignores the associated boolean flag indicating if it is active. Consequently, a submitter whose privileges have been revoked (`active = false`) can still successfully call `approve_token_transfer`, defeating the revocation mechanism entirely.

The `update_submitter` function correctly sets the active status for a given submitter address in the `inner.submitters` map.

```
public fun update_submitter(
    bridge: &mut Bridge,
    admin_cap: &AdminCap,
    submitter: address,
    active: bool,
) {
    assert_admin_version(admin_cap);
    let inner = load_inner_mut(bridge);
    if (inner.submitters.contains(&submitter)) {
        let _active = inner.submitters.get_mut(&submitter);
        *_active = active;
    } else {
        inner.submitters.insert(submitter, active);
    };
    emit(UpdateBridgeSubmitter{submitter, active});
}
```

However, the `approve_token_transfer` function only checks for the presence of the sender's address as a key in the map, neglecting to check its value.

```
public fun approve_token_transfer(
    bridge: &mut Bridge,
    message: BridgeMessage,
```

```

signatures: vector<vector<u8>>,
ctx: &TxContext,
) {
let inner = load_inner_mut(bridge);
assert!(!inner.paused, EBridgeUnavailable);

// verify sender is a submitter
assert!(inner.submitters.contains(&ctx.sender()), ESenderNotSubmitter);

// verify signatures
inner.committee.verify_signatures(message, signatures);

assert!(message.message_type() == message_types::token(), EMustBeTokenMessage
);
assert!(message.message_version() == MESSAGE_VERSION, EUnexpectedMessageVersi
on);
let token_payload = message.extract_token_bridge_payload();
let target_chain = token_payload.token_target_chain();
assert!(
message.source_chain() == inner.chain_id || target_chain == inner.chain_id,
EUnexpectedChainID,
);

let message_key = message.key();
// retrieve pending message if source chain is Iota, the initial message
// must exist on chain
if (message.source_chain() == inner.chain_id) {
let record = &mut inner.token_transfer_records[message_key];

assert!(record.message == message, EMalformedMessageError);
assert!(!record.claimed, EInvariantIotaInitializedTokenTransferShouldNotBeCla
imed);

// If record already has verified signatures, it means the message has been a
proved
// Then we exit early.
if (record.verified_signatures.is_some()) {
emit(TokenTransferAlreadyApproved { message_key });
return
};
// Store approval
record.verified_signatures = option::some(signatures)
} else {

```

```
// At this point, if this message is in token_transfer_records, we know
// it's already approved because we only add a message to token_transfer_reco
rds
// after verifying the signatures
if (inner.token_transfer_records.contains(message_key)) {
  emit(TokenTransferAlreadyApproved { message_key });
  return
};
// Store message and approval
inner.token_transfer_records.push_back(
  message_key,
  BridgeRecord {
    message,
    verified_signatures: option::some(signatures),
    claimed: false
  },
);
};

emit(TokenTransferApproved { message_key });
}
```

The problem lies in this authorization logic. The `vec_map::contains()` function only verifies that the given key (`ctx.sender()`) is present in the map; it does not inspect the boolean value associated with that key. When an administrator revokes a submitter's privileges by setting their status to false, their address remains as a key in the submitters map. As a result, the authorization check in `approve_token_transfer` will still pass for this "disabled" submitter, incorrectly treating them as fully authorized and rendering the revocation mechanism ineffective.

This issue renders the submitter revocation process ineffective. An administrator might believe they have successfully blocked a compromised, malicious, or retired submitter, but that entity would retain its full privileges to call `approve_token_transfer`.

Assets:

- `./bridge.move` [<https://github.com/echo-proto/bridge-iota>]

Status:

Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Modify the check in `approve_token_transfer` to verify both the existence of the submitter key and that its associated boolean value is true.

Resolution: Fixed in `5d53b26`, associated to submitter boolean value is validating:

```
assert!(*inner.submitters.get(&ctx.sender()), ESenderNotSubmitter);
```


F-2025-12021 - Artificially Low Asset Price Leads to Rate-Limiter Bypass - Medium

Description:

The `bridge` contract uses a rate-limiting mechanism (`limiter.move`) to cap the total USD value of tokens that can be transferred out per hour for a given route (e.g., ETH from Ethereum). To calculate the value of an outbound transfer, the `limiter` multiplies the token amount by its `notional_value`, which is meant to be the token's USD price scaled by a multiplier (`USD_VALUE_MULTIPLIER = 1e8`). Prices are set by the governance committee via `UpdateAssetPrice` system messages.

The `notional_value` used for price calculations is not validated, allowing governance to mistakenly or maliciously set a token's price to an extremely low value (e.g., 1 instead of price * 1e8). Because the rate-limiter uses this value to enforce its USD-denominated caps, an artificially low price effectively neutralizes the transfer limit. An attacker could exploit this to drain bridge funds by transferring far more value than the route's configured hourly limit.

The `check_and_record_sending_transfer` function in `limiter.move` is responsible for enforcing hourly transfer limits. It calculates the notional USD value of a transfer with the following logic:

```
let value = (treasury.notional_value<T>() as u128);
let notional_amount_with_token_multiplier = value * (amount as u128);
let notional_amount = notional_amount_with_token_multiplier / (treasury.decimal_multiplier<T>() as u128);
```

The `notional_value<T>()` is fetched directly from the treasury module. The issue is that the function for setting this price, `update_asset_notional_price`, does not validate its input. It is called by `execute_update_asset_price` in `bridge.move`, which is in turn only callable via a committee-signed `execute_system_message`.

```
public(package) fun update_asset_notional_price(
    self: &mut BridgeTreasury,
    token_id: u8,
    new_usd_price: u64,
) {
    let type_name = self.id_token_type_map.try_get(&token_id);
    assert!(type_name.is_some(), EUnsupportedTokenType);
    assert!(new_usd_price > 0, EInvalidNotionalValue);
    let type_name = type_name.destroy_some();
    let metadata = self.supported_tokens.get_mut(&type_name);
```

```
metadata.notional_value = new_usd_price;
}
```

As the provided test case `test_limit_bypass_via_underpricing` demonstrates, if governance sets BTC's `notional_value` to 1, a transfer that should be worth over \$50,000 USD is instead valued at \$0.00000001. A route with a \$10 hourly limit, which would correctly block the transfer, instead approves it. This vulnerability applies to both updating existing token prices and adding new tokens.

If the committee sets an artificially low price for a high-value asset, the hourly transfer limits for that asset become ineffective.

Assets:

- `./limiter.move` [<https://github.com/echo-proto/bridge-iota>]

Status:

Accepted

Classification

Impact Rate:	5/5
Likelihood Rate:	3/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: Modify `update_asset_notional_price` to perform basic validation on the `new_usd_price`. At a minimum, ensure the price is not drastically different from a reasonable floor, such as the `USD_VALUE_MULTIPLIER`.

Resolution: The client has acknowledged the finding with the following statement:

I don't consider this to be an issue, we'll not fix it.

Evidences

PoC

Reproduce:

1. A transfer route for BTC is configured with a very low hourly limit, like \$10. This is the safety rail we expect to function.
2. The test attempts to send a small amount of BTC. With BTC's price correctly set to ~\$50,000, this transfer is worth far more than the \$10 limit.
3. The test asserts that the transfer is correctly **rejected**. The limiter works as expected, returning false because the value exceeds the cap.
4. The test then simulates a governance error by updating BTC's `notional_value` to 1. This tricks the contract into thinking one BTC is worth only a fraction of a cent.
5. The exact same transfer is attempted again. This time, because the price is artificially low, the transfer's calculated value is almost zero and easily fits within the \$10 limit. The test asserts the transfer is now **accepted**, proving the limiter has been bypassed.

PoC Code:

```
#[test_only]
module bridge::notional_price_limit_bypass_test {
  use bridge::bridge::{
    new_for_testing,
    test_load_inner_mut,
    inner_limiter_mut,
  };
  use bridge::chain_ids;
  use bridge::message;
  use bridge::treasury::BTC;
  use iota::clock::{Self as Clock};
  use iota::test_scenario;
  use iota::test_utils::destroy;

  const UNEXPECTED: u64 = 424242;

  #[test]
  fun test_limit_bypass_via_underpricing() {
    let sender = @0x0;
    let mut scenario = test_scenario::begin(sender);
    let ctx = scenario.ctx();
    let chain_id = chain_ids::chain_id();

    let mut bridge = new_for_testing(chain_id, ctx);
    let inner = bridge.test_load_inner_mut();
```

```

let btc_id: u8 = 1;
let sending_chain: u8 = 77;
let add_route_msg = message::create_add_routes_on_iota_message(
    chain_id,
    0, // seq_num
    vector[sending_chain],
    vector[btc_id],
    vector[0], // fee percentage
    vector[0], // initial bridge amount
    vector[true],
);
inner.test_execute_add_routes(message::extract_add_routes_on_iota(&add_route_
msg));

let route = chain_ids::get_route(inner.inner_routes(), sending_chain, btc_id)
;

inner_limiter_mut(inner).update_route_limit(&route, 10);

let mut clk = Clock::create_for_testing(ctx);
clk.set_for_testing(123_456_000); // arbitrary timestamp

let transfer_ok = inner.test_check_and_record_transfer<BTC>(&clk, route, 1 /*
amount = 1 satoshi */);
assert!(!transfer_ok, UNEXPECTED);

inner.inner_treasury_mut().update_asset_notional_price(btc_id, 1);
let transfer_ok_after_cheat = inner.test_check_and_record_transfer<BTC>(&clk,
route, 1);
assert!(transfer_ok_after_cheat, UNEXPECTED);

clk.destroy_for_testing();
destroy(bridge);
scenario.end();
}
}

```

Results:

```
[ PASS ] bridge::notional_price_limit_bypass_test::test_limit_bypass_via_u
nderpricing
Test result: OK. Total tests: 1; passed: 1; failed: 0
Total number of linter warnings suppressed: 1 (unique lints: 1)
```

F-2025-11854 - Token ID Collision Allows State Corruption - Low

Description:

The `add_new_token` function is a privileged, `public(package)` function intended for administrative use. Its purpose is to approve a new foreign token by moving it from the `waiting_room` to the list of `supported_tokens`. It assigns a `token_id` to the token's `TypeName` and creates two mappings: one from `TypeName` to `metadata` (which includes the ID) and another from the `token_id` back to the `TypeName`.

The `add_new_token` function lacks a validation to ensure that a new `token_id` is unique before it's assigned to a token. An administrator could inadvertently (or maliciously) assign an already-used `token_id` to a new token. This would overwrite the entry in the `id_token_type_map`, causing the contract's state to become inconsistent. As a result, the previously registered token becomes "orphaned" and unmanageable through ID-based functions, while functions relying on this mapping for validation will behave incorrectly.

When an administrator calls `add_new_token`, the function takes a `token_id` as an argument. The contract uses two different data structures to track tokens:

1. `supported_tokens`: A `VecMap<TypeName, BridgeTokenMetadata>`
2. `id_token_type_map`: A `VecMap<u8, TypeName>`

The issue exists because there is no check to see if `id_token_type_map` already contains the `token_id` being added. The insert operation will simply overwrite any existing value for that key.

```
public(package) fun add_new_token(
    self: &mut BridgeTreasury,
    token_name: String,
    token_id: u8,
    // ...
) {
    if (!native_token){
        // ...
        let ForeignTokenRegistration{ type_name, decimal } = self.waiting_room
        .remove<String, ForeignTokenRegistration>(token_name);

        if (!self.supported_tokens.contains<TypeName, BridgeTokenMetadata>(&ty
pe_name)) {
            self.supported_tokens.insert(
                type_name,
                BridgeTokenMetadata{ id: token_id, /* ... */ },
            );
        }
    }
}
```

```
};
self.id_token_type_map.insert(token_id, type_name);
// ...
}
}
```

This can lead to:

- The original token associated with the overwritten `token_id` can no longer be managed by any function that relies on the `id_token_type_map` for lookups. This includes critical functions like `update_asset_notional_price`. The asset's price would become permanently fixed, which could be exploited in other parts of the system.
- The contract's internal state becomes contradictory.
- Functions like `is_valid_token_id` would return incorrect results.

Assets:

- `./treasury.move` [<https://github.com/echo-proto/bridge-iota>]

Status:

Accepted

Classification

Impact Rate:	3/5
Likelihood Rate:	3/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation: Add an assertion to `add_new_token` to verify that the `token_id` is not already in use before inserting it into `id_token_type_map`.

Resolution: The client has acknowledged the finding with the following statement:

This is by design — it allows overwriting the current `token_id` and `token_type`.

[F-2025-11904](#) - Unchecked Read in peel_u64_be Leads to Transaction Abort and Denial of Service - Low

Description:

The `message` module defines various cross-chain message formats. Several of these formats include `u64` fields for values like sequence numbers, token amounts, and price limits (e.g., `TokenTransferPayload`, `UpdateBridgeLimit`). The `peel_u64_be` function is a custom deserializer used to read these 8-byte, big-endian encoded integers from a raw message payload during on-chain processing. It is a critical component in parsing messages that are submitted to the bridge for approval and execution.

The `peel_u64_be` function unconditionally attempts to read 8 bytes from the provided byte stream without first verifying that 8 bytes are available. A malicious actor can craft and submit a message with a truncated payload, causing the function to attempt a read past the buffer's end. This triggers a transaction abort, which can be exploited to create a Denial-of-Service (DoS) attack, preventing any valid messages of that type from being processed by the bridge.

The vulnerability lies in the implementation of `peel_u64_be` in `message.move`. The function contains a while loop that is hardcoded to execute exactly eight times to read eight bytes, but it does so without any bounds checking.

```
fun peel_u64_be(bcs: &mut BCS): u64 {
  let (mut value, mut i) = (0u64, 64u8);
  while (i > 0) {
    i = i - 8;
    let byte = (bcs::peel_u8(bcs) as u64);
    value = value + (byte << i);
  };
  value
}
```

This unsafe function is used to deserialize critical fields in other parsing functions. An attacker can exploit this by constructing a `TokenTransfer` message where the payload is deliberately incomplete.

The issue allows for a **Denial-of-Service (DoS)** vulnerability against specific bridge functionalities. Repeatedly submitted malformed messages, once approved by a compromised or unaware committee, will cause any transaction attempting to process them to fail.

Assets:

- ./message.move [<https://github.com/echo-proto/bridge-iota>]

Status:**Accepted**

Classification**Impact Rate:** 3/5**Likelihood Rate:** 2/5**Exploitability:** Independent**Complexity:** Simple**Severity:** **Low**

Recommendations

Remediation: The `peel_u64_be` function must validate that it has sufficient bytes to read before entering its processing loop. Add an assertion at the beginning of the function to check that at least 8 bytes are remaining in the `bcs` stream.

Resolution: The client has acknowledged the finding with the following statement:

First, the code here cannot access `bcs.bytes`, as it will cause a compilation error. Adding this `assert` has no real significance, so we have decided not to make any changes.

[F-2025-11995](#) - Lack of Zero-Value Validation in `update_route_limit()` Allows Setting Invalid Rate Limits - Low

Description:

The `bridge` module enables bridging assets from an EVM to IOTA. To prevent abuse, it enforces a per-route hourly transfer rate limit. If a transfer exceeds this limit, it is rejected, and a `TokenTransferLimitExceed` event is emitted.

```
// Claim token from approved bridge message
// Returns Some(Coin) if coin can be claimed. If already claimed, return None
fun claim_token_internal<T>(
    bridge: &mut Bridge,
    clock: &Clock,
    source_chain: u8,
    bridge_seq_num: u64,
    ctx: &mut TxContext,
): (Option<Coin<T>>, address) {
    ...

    let amount = token_payload.token_amount();
    // Make sure transfer is within limit.
    if (!inner
        .limiter
        .check_and_record_sending_transfer<T>(
            &inner.treasury,
            clock,
            route,
            amount,
        )
    ) {
        emit(TokenTransferLimitExceed { message_key: key });
        return (option::none(), owner)
    };
    ...
}
```

Authorized committee members can update this limit by submitting a signed message via the `execute_system_message()` function. Once the message reaches the required voting power and is verified, the `update_route_limit()` function is called to apply the new rate limit.

However, the `update_route_limit()` function does not validate the `new_usd_limit` input parameter. As a result, it is possible to set the rate limit to zero.

```

public(package) fun update_route_limit(
    self: &mut TransferLimiter,
    route: &BridgeRoute,
    new_usd_limit: u64
) {
    let sending_chain = *route.destination();
    let sending_token = *route.token();

    if (!self.transfer_limits.contains(route)) {
        self.transfer_limits.insert(*route, new_usd_limit);
    } else {
        *&mut self.transfer_limits[route] = new_usd_limit;
    };

    emit(UpdateRouteLimitEvent {
        sending_chain: sending_chain,
        sending_token: sending_token,
        new_limit: new_usd_limit,
    })
}

```

The module's logic does not expect this case, and treats a zero limit as a valid configuration. Consequently, all subsequent transfers on the affected route will fail with `TokenTransferLimitExceed`, effectively disabling bridging for that route until the limit is updated again.

```

public(package) fun check_and_record_sending_transfer<T>(
    self: &mut TransferLimiter,
    treasury: &BridgeTreasury,
    clock: &Clock,
    route: BridgeRoute,
    amount: u64
): bool {
    ...

    // Check if transfer amount exceed limit
    // Upscale them to the token's decimal.
    if ((record.total_amount as u128)
        * (treasury.decimal_multiplier<T>() as u128)
        + notional_amount_with_token_multiplier > route_limit_adjusted // rou
te_limit_adjusted = 0
    ) {
        return false
    };
    ...
}

```

Setting a route's rate limit to zero disables all future transfers on that route, even if the route is otherwise valid. This can unintentionally block users from bridging their assets and disrupt protocol usability until the issue is manually corrected by the committee.

Assets:

- `./limiter.move` [<https://github.com/echo-proto/bridge-iota>]

Status:

Accepted

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations**Remediation:**

Consider adding input validation to the `update_route_limit()` function to ensure that the `new_usd_limit` value is greater than zero:

```
assert!(new_usd_limit > 0, EInvalidLimitValue);
```

Resolution:

The client has acknowledged the finding with the following statement:

This is by design — since the default value is 0, modification to 0 is also allowed.

F-2025-11855 - Inconsistent Logic Due to Partial Redundancy

Check - Info

Description:

The `add_new_token` function is a privileged function used to approve a new foreign token. It populates two data structures, `supported_tokens` (mapping a token's `TypeName` to its metadata) and `id_token_type_map` (mapping a `token_id` back to a `TypeName`).

The function includes a check to prevent overwriting an entry in `supported_tokens` if the token's `TypeName` already exists. However, it fails to apply the same conditional logic to the `id_token_type_map`. This logical inconsistency allows for partial updates to the contract's state, leading to a mismatch between the two data structures and creating a subtle but problematic form of state corruption.

The code explicitly checks if a token `type_name` is already present in `supported_tokens`. If it is, the insert operation is skipped. However, the subsequent insertion into `id_token_type_map` is performed unconditionally, regardless of the outcome of the previous check.

```
if (!self.supported_tokens.contains<TypeName, BridgeTokenMetadata>(&type_name)) {
    self.supported_tokens.insert(
        type_name,
        BridgeTokenMetadata{
            id: token_id,
            //...
        },
    );
    self.id_token_type_map.insert(token_id, type_name);
}
```

This creates a confusing and inconsistent state that complicates contract management and can lead to logic errors.

Assets:

- `./treasury.move` [<https://github.com/echo-proto/bridge-iota>]

Status:

Fixed

Classification

Impact Rate:

1/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: To ensure atomicity and consistency, the insertion into `id_token_type_map` should be moved inside the if block. This ensures that both data structures are updated together if and only if the token is genuinely new.

Resolution: Fixed in `d60e18e`, the `if` block was removed to ensures that both data structures are updated together.

[F-2025-11994](#) - Commented Code Parts - Info

Description:

The protocol contains commented-out sections across multiple modules. These include commented import statements, functions, storage variables, and assert validations within certain function implementations. Such commented logic may indicate partially implemented or deprecated functionality that was never fully removed from the codebase.

The presence of commented-out code creates ambiguity about the intended functionality and the current state of the system. It undermines code readability, reduces maintainability, and complicates future upgrades. Moreover, it may reflect uncertainty in the protocol's logic and increase the risk of inconsistent assumptions during ongoing development.

Additionally, there are `TODO` and `FIXME` comments that indicate areas requiring attention or completion. While such comments can be useful during the development phase, leaving them in production code may pose risks.

```
fun execute_add_tokens_on_iota(inner: &mut BridgeInner, payload: AddTokenOnIota) {
    // FIXME: assert native_token to be false and add test
    let native_token = payload.is_native();
    ...
}

...

fun execute_add_routes_on_iota(inner: &mut BridgeInner, payload: AddRoutesOnIota) {
    // FIXME: assert native_token to be false and add test
    // let native_token = payload.is_native();
    let mut chain_ids = payload.supported_chain_ids();
    ...
}

...

public fun execute_system_message(
    bridge: &mut Bridge,
    message: BridgeMessage,
    signatures: vector<vector<u8>>,
) {
    let message_type = message.message_type();

    // TODO: test version mismatch
```

```
assert!(message.message_version() == MESSAGE_VERSION, EUnexpectedMessageVersion);  
  
...  
}
```

Assets:

- ./bridge.move [https://github.com/echo-proto/bridge-iota]
- ./message.move [https://github.com/echo-proto/bridge-iota]
- ./treasury.move [https://github.com/echo-proto/bridge-iota]
- ./committee.move [https://github.com/echo-proto/bridge-iota]
- ./limiter.move [https://github.com/echo-proto/bridge-iota]
- ./chain_ids.move [https://github.com/echo-proto/bridge-iota]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider removing all commented-out code unless it serves a clearly documented and justified purpose, and address all `TODO` and `FIXME` comments. If any portion of the commented code is intended for future use, it should be managed through version control systems or detailed developer documentation, but not left in production source files.

Resolution: Fixed in `d2b7f09`, all commented parts of the code were deleted.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/echo-proto/bridge-iota
Initial Commit	686e118
Final Commit	89ffe35
Whitepaper	N/A
Requirements	https://github.com/echo-proto/bridge-iota
Technical Requirements	https://github.com/echo-proto/bridge-iota

Asset	Type
./bridge.move [https://github.com/echo-proto/bridge-iota]	Smart Contract
./chain_ids.move [https://github.com/echo-proto/bridge-iota]	Smart Contract
./committee.move [https://github.com/echo-proto/bridge-iota]	Smart Contract
./crypto.move [https://github.com/echo-proto/bridge-iota]	Smart Contract
./ibtc.move [https://github.com/echo-proto/bridge-iota]	Smart Contract
./limiter.move [https://github.com/echo-proto/bridge-iota]	Smart Contract
./message_types.move [https://github.com/echo-proto/bridge-iota]	Smart Contract
./message.move [https://github.com/echo-proto/bridge-iota]	Smart Contract
./treasury.move [https://github.com/echo-proto/bridge-iota]	Smart Contract

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.